

Assignment #2: Event-Driven System Implementation

Maze Game

Overview

The game is a simple 3D maze game where you navigate through a maze avoiding the walls. If you collide with a wall you take damage and your health bar decreases. The project includes health treasure chest that is triggered after the player collects coloured cubes.

System Design & Event Flow

- Player Movement: You control the player with WASD keys using Rigidbody and AddForce Y-axis movement is locked so you stay on the ground
- Collision Detection: OnCollisionEnter detects when you have hit an object tagged "Wall"
- Health System: When a collision happens, TakeDamage() reduces your health and updates on the Health Bar

Events & Modularity

The system is Dissociated, meaning the pieces of the game talk to each other without being permanently stuck together.

- The Interface (IInteractable): It allows the Player to walk up to a Chest, a Door, or a Key and use the same 'E' key for all of them without needing separate code for every single object.
- The Event Provider (TreasureChest): The chest doesn't know the UI exists. It just shouts a signal (onChestOpened) into the air when it's opened. This is using UnityEngine.Events.
- The Event Listener (MazeGameManager): This script sits on your UI Canvas. It is listening for that specific chest signal. When it hears the signal, it updates the health bar automatically.
- The Logic Gate (Inventory): Before the chest is allowed to shout its signal, it checks a Gate. It asks the PlayerInventory script: "Does the player have the Red Cube and the Green Cube?" If yes, the event fires.

Challenges & Solutions

- Collision issues but fixed as soon as I made sure there was a Rigidbody and Collider
- The Health Bar was not updating and forgot to add the player script to the player which included the OnCollisionEnter
- Floating player when hitting W, fixed using Rigidbody constraints to freeze vertical movement
- Items like the green and red cube were empty 3D models, but resolved by adding box colliders set to Is Trigger and an OnTriggerEnter script to update player inventory

Conclusion

By utilizing Interfaces and **UnityEvents**, the maze game demonstrates a flexible approach where gameplay objects and UI elements are independent. This gives a flexible, event-driven architecture that is easy to expand with new interactive features